COMPUTACIÓN Y ESTRUCTURAS DISCRETAS III · INTEGRATIVE TASK 1

ResumeLens

Formal language-based résumé screening

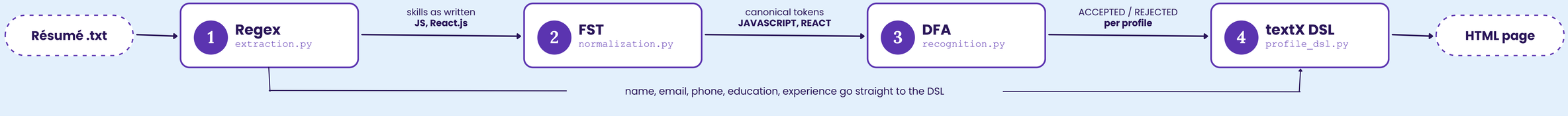
Jhoan Manuel Tovar Rendón · Juan Pablo Martinez Rosero · Jhon Edwin Escudero
Cali, Colombia · 2026-2

Universidad ICESI

CENTRAL QUESTION

Can formal language models decide, in an explainable way, whether the skills written in a résumé match a professional profile?

Figure 1. One résumé through the four stages.



The problem

The same skill is written in many ways: **JS**, **Javascript**, **JavaScript**. ResumeLens checks whether the skills in a résumé match a profile pattern. It does not rank candidates.

Four profiles

Full Stack Developer
JS/TS, frontend, backend, database, Git
ML Engineer
Python, Pandas/NumPy, ML library, Git
DevOps Engineer
Linux, Docker, CI/CD, cloud, Git
Data Engineer
Python, SQL, Spark, Airflow, Git

Design decisions

- **Abstraction.** A skill is one word-level symbol, not a string of characters.
- **Alphabet.** Σ is the set of 49 canonical tokens. The FST input is the variant words plus the end mark #.
- **DFA, not NFA.** After sorting, each profile has a single path, so 7 to 9 states are enough.
- **Grammar.** The DSL follows the parts of a résumé: contact, education, experience, skills, accepted profiles.
- **Criterion.** A profile is accepted only when its DFA accepts the sorted tokens.

STAGE 1 · DETECTION

1 Regular expressions

One pattern per data type. Skills are read only from the **Skills:** section. Boundaries stop **JS** from matching inside **React.js**.

```
(?<![\w.])(JavaScript|JS|
  TypeScript|Python|...)(?!\w)
```

Categories run in a fixed order and erase what they found, so **SQL** inside **SQL Server** is not counted twice.

STAGE 2 · TRANSFORMATION

2 Finite-state transducers

Five FSTs, 49 canonical tokens, 114 variants. Every spelling of a skill ends in the same state, and the end mark **#** writes the token.

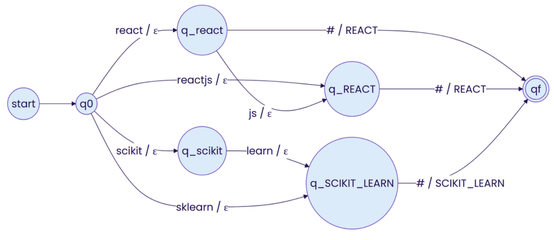


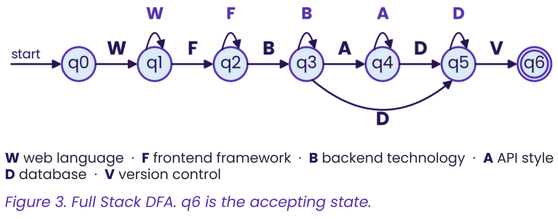
Figure 2. Excerpt of T_FW. Label = input word / output.

```
React.js  →  react js #  →  REACT
```

STAGE 3 · CLASSIFICATION

3 Finite automata

Tokens are sorted in each profile's order. One DFA per profile: one state per group, jumps only over optional groups.



W web language · **F** frontend framework · **B** backend technology · **A** API style
D database · **V** version control

Figure 3. Full Stack DFA. q6 is the accepting state.

STAGE 4 · VALIDATION

4 Context-free grammar

A textX language describes the candidate. Lexical and syntax errors are rejected with line and column.

```
candidate "Wednesday Addams" {
  skills { JAVASCRIPT, REACT, ... }
  accepted { profile FULL_STACK_DEVELOPER
    : JAVASCRIPT, REACT, ... } }
```

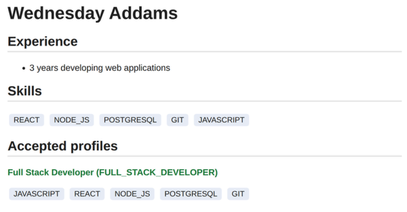


Figure 4. HTML page generated from a valid model.

Results

Résumé	Accepted profile
Wednesday Addams	Full Stack
Mary Jane Watson	ML Engineer
Leon Kennedy	Full Stack
Chris Redfield	DevOps
Catherine Halsey	Data Engineer
Ada Wong	ML + Data Engineer
Jill Valentine	*none*
Avery Johnson	*none*

Table 1. Accepted profiles for the 8 test résumés. none = a required group is missing.

65
automated tests pass

2/2
assignment examples correct

Limitations

Only skills under a **Skills:** heading are read. Typos and unlisted spellings are not recognized.

Conclusion

Each formal model fits one task: find strings, rewrite them, recognize patterns, validate structure. That is why every result can be explained.

References Hopcroft, Motwani & Ullman (2006), Introduction to Automata Theory · Mohri (1997), Computational Linguistics 23(2) · Romero (2021), Pyformlang, SIGCSE · Dejanović et al. (2017), textX, Knowledge-Based Systems 115

Repository [https://github.com/JhoanToovar/ResumeLens]